



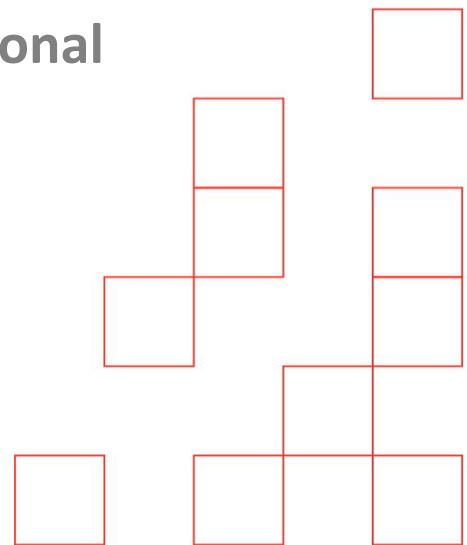
**Universidad
Europea**



Proyecto Integrador: Gestión tienda

Documento Funcional

2025-2026



DAM

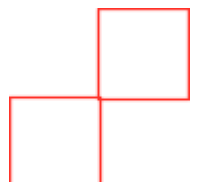
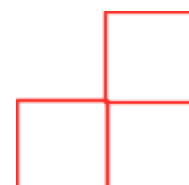


Tabla de contenido

DAM	1
I. Introducción	3
Objetivo del documento:	3
Alcance del proyecto:	3
Descripción general de la aplicación	3
II. Descripción del sistema	3
Nombre de la aplicación: 'GestionTienda'	3
Contexto (tienda, concesionario, librería, etc.): Tienda del estilo de FNAC.//TODO explicar un poco más	3
Usuarios objetivo: Administradores, empleados y compradores de la tienda.	4
III. Requisitos funcionales	4
RF1: El sistema permitirá dar de alta un producto	4
RF2: El sistema permitirá consultar productos	4
RF3: El sistema permitirá modificar productos	4
RF4: El sistema permitirá eliminar productos	
RF5: El sistema permitirá dar de alta empleados	
RF6: El sistema permitirá eliminar empleados	
RF7: El sistema permitirá agregar productos a un carrito de compra	
RF8: El sistema permitirá eliminar productos del carrito de compra	
RF9: El sistema permitirá agregar stock de un producto	
RF10: El sistema permitirá eliminar stock de un producto	
RF11: El sistema permitirá realizar un simulacro de pago	4
IV. Requisitos no funcionales	4
V. Casos de uso	5
Caso de uso 1: Ver productos	5
VI. Modelo de datos	9
VII. Arquitectura de la aplicación	12
VIII. Equipo	14
IX. Observaciones	14



I. Introducción

Este documento forma parte de la documentación del Trabajo Integrador de 1º de DAM, cuyo objetivo es poner en práctica los contenidos aprendidos durante el curso. Concretamente, en el módulo de Programación con el desarrollo de una aplicación funcional en Java con interfaz hecha en Java Swing. En el módulo de Bases de Datos se implementa la persistencia mediante SQLite con consultas SQL. Además, el proyecto integra el uso de herramientas de testing, tanto de caja blanca como de caja negra, y el control de versiones con Git.

Objetivo del documento:

El objetivo de este documento es esclarecer el funcionamiento de nuestra aplicación de trabajo integrador, orientada al manejo de una tienda de estilo FNAC. Asimismo, se desglosan cada una de sus partes junto con una explicación detallada, dirigida a los evaluadores.

Alcance del proyecto:

Nuestra aplicación abarca tres ramas de funcionalidades orientadas a tres tipos distintos de usuarios, empleados, compradores y administradores.

Por el lado de los empleados estos podrán administrar el stock, agregar y eliminar la cantidad disponible de cada producto.

Los compradores podrán ver el listado de productos y agregarlos, en la cantidad que quieran a su carrito de compra. Una vez satisfechos con su compra, en la ventana de carrito podrán ver su carrito con la lista de productos y eliminar el que deseen en el caso de así quererlo; por otro lado habrá un simulacro de pago.

Los administradores podrán registrar y eliminar trabajadores, agregar y eliminar productos y ver el listado de “transacciones” realizadas detallando qué comprador la realizó y si hubo un empleado que lo ayudó y el importe de esta.

La limitación que tenemos con respecto a los compradores es que la aplicación no puede manejar transacciones verdaderas.

Descripción general de la aplicación

Es una aplicación de escritorio orientada a la gestión básica de una tienda. Se desarrolla utilizando java swing para la interfaz gráfica siguiendo un patrón de diseño mvc y sqlite para el manejo de la base de datos.

Cuenta con tres tipos de usuarios, administrador, empleado y comprador, cada uno con ventanas y flujos propios.

Por el lado de las herramientas a utilizar son github para el manejo del trabajo en equipo y versiones, Eclipse como entorno de desarrollo principal, DB browser para visualizar y manejar la base de datos y draw.io para la creación de los diagramas.

Por último su ejecución será desde un jar.

II. Descripción del sistema



Nombre de la aplicación: 'GestionTienda'

Contexto (tienda, concesionario, librería, etc.): Va a ser una aplicación para gestionar una tienda de cualquier índole pudiendo en un futuro ser personalizada a pedido del cliente para que se adecúe a su tipo de tienda.

Usuarios objetivo: Administradores, empleados y compradores de la tienda.

III. Requisitos funcionales

RF1: El sistema permitirá dar de alta un producto

RF2: El sistema permitirá consultar productos

RF3: El sistema permitirá modificar productos

RF4: El sistema permitirá eliminar productos

RF5: El sistema permitirá dar de alta empleados

RF6: El sistema permitirá eliminar empleados

RF7: El sistema permitirá agregar productos a un carrito de compra

RF8: El sistema permitirá eliminar productos del carrito de compra

RF9: El sistema permitirá agregar stock de un producto

RF10: El sistema permitirá eliminar stock de un producto

RF11: El sistema permitirá realizar un simulacro de pago

RF12: El sistema permitirá dar de alta un comprador

RF13: El sistema permitirá dar de baja un comprador

RF14: El sistema permitirá modificar un comprador

IV. Requisitos no funcionales



- Interfaz gráfica intuitiva
- Tiempo de respuesta aceptable
- Persistencia en base de datos relacional
- Seguridad básica (validación de datos)
- Portabilidad
- Control de acceso por roles
- Independencia de internet

V. Casos de uso

Caso de uso 1: Ver productos

- Actor: Administrador
- Descripción: Permite ver el listado de productos
- Flujo principal:
 1. Iniciar sesión
 2. El sistema valida
 3. Muestra el listado de productos y opciones para ejecutar los flujos secundarios
- Flujo alternativo:
 - Inicio de sesión inválido → mensaje de error
 - Agregar un producto
 - Eliminar producto
 - Modificar producto

Caso de uso 1.1: Agregar un producto

- Descripción: Agrega un producto a la base de datos
- Flujo principal:
 - Introducir datos
 - Validar datos
 - Producto añadido
- Flujo alternativo:
 - Datos inválidos → Mostrar mensaje de error

Caso de uso 1.2: Eliminar producto

- Descripción: Eliminar producto de la base de datos
- Flujo principal:



- Seleccionar producto de la lista
- Eliminar producto
- Flujo alternativo:
 - Producto sin seleccionar → Mensaje de error

Caso de uso 1.3: Modificar producto

- Descripción: Modificar datos de un producto dentro de la base de datos
- Flujo principal:
 - Introducir datos
 - Validar datos
 - Modificar producto
- Flujo alternativo:
 - Datos inválidos → Mostrar mensaje de error

Caso de uso 2: Ver productos y su stock

- Actor: Empleado
- Descripción: Permite ver el listado de productos y cuánto stock disponible tienen
- Flujo principal:
 - Iniciar sesión
 - El sistema valida
 - Muestra el listado de productos junto con su stock y opciones para ejecutar los flujos secundarios
- Flujo alternativo
 - Inicio de sesión inválido → mensaje de error
 - Agregar stock
 - Eliminar stock

Caso de uso 2.1: Agregar stock

- Descripción: Añade stock disponible de un producto en la base de datos
- Flujo principal:
 - Seleccionar la cantidad a añadir
 - Añadir stock

Caso de uso 2.2: Eliminar stock

- Descripción: Eliminar stock disponible de un producto en la base de datos
- Flujo principal:
 - Seleccionar la cantidad a eliminar
 - Eliminar stock

Caso de uso 3: Registrarse

- Actor: Comprador
- Descripción: Te permite crearte un usuario como comprador
- Flujo principal:
 - Introducir datos



- El sistema valida
- Creación de cuenta
- Flujo alternativo:
 - Datos inválidos→ mensaje de error

Caso de uso 4: Ver productos

- Actor: Comprador
- Descripción: Te permite ver el listado de productos
- Flujo principal:
 - Iniciar sesión
 - El sistema valida
 - Muestra el listado de productos y opciones para ejecutar los flujos secundarios
- Flujo alternativo:
 - Inicio de sesión inválido→ mensaje de error
 - Agregar producto al carrito
 - Eliminar producto del carrito

Caso de uso 4.1: Agregar producto al carrito

- Descripción: Agregar un producto al carrito de compra
- Flujo principal:
 - Seleccionar producto y cantidad a añadir
 - Añadir producto y cantidad al carrito

Caso de uso 4.2: Eliminar producto del carrito

- Descripción: Eliminar producto del carrito de compra
- Flujo principal:
 - Seleccionar producto a eliminar
 - Eliminar producto
- Flujo alternativo:
 - Producto no seleccionado →Mensaje de error

Caso de uso 5: Ver carrito

- Actor: Comprador
- Descripción: Permite ver el listado de productos agregados al carrito
- Flujo principal:
 - Iniciar sesión
 - El sistema valida
 - Muestra el listado de productos agregados al carrito y opciones para ejecutar los flujos secundarios
- Flujo alternativo:
 - Inicio de sesión inválido→ mensaje de error
 - “Pagar”

- Eliminar producto del carrito

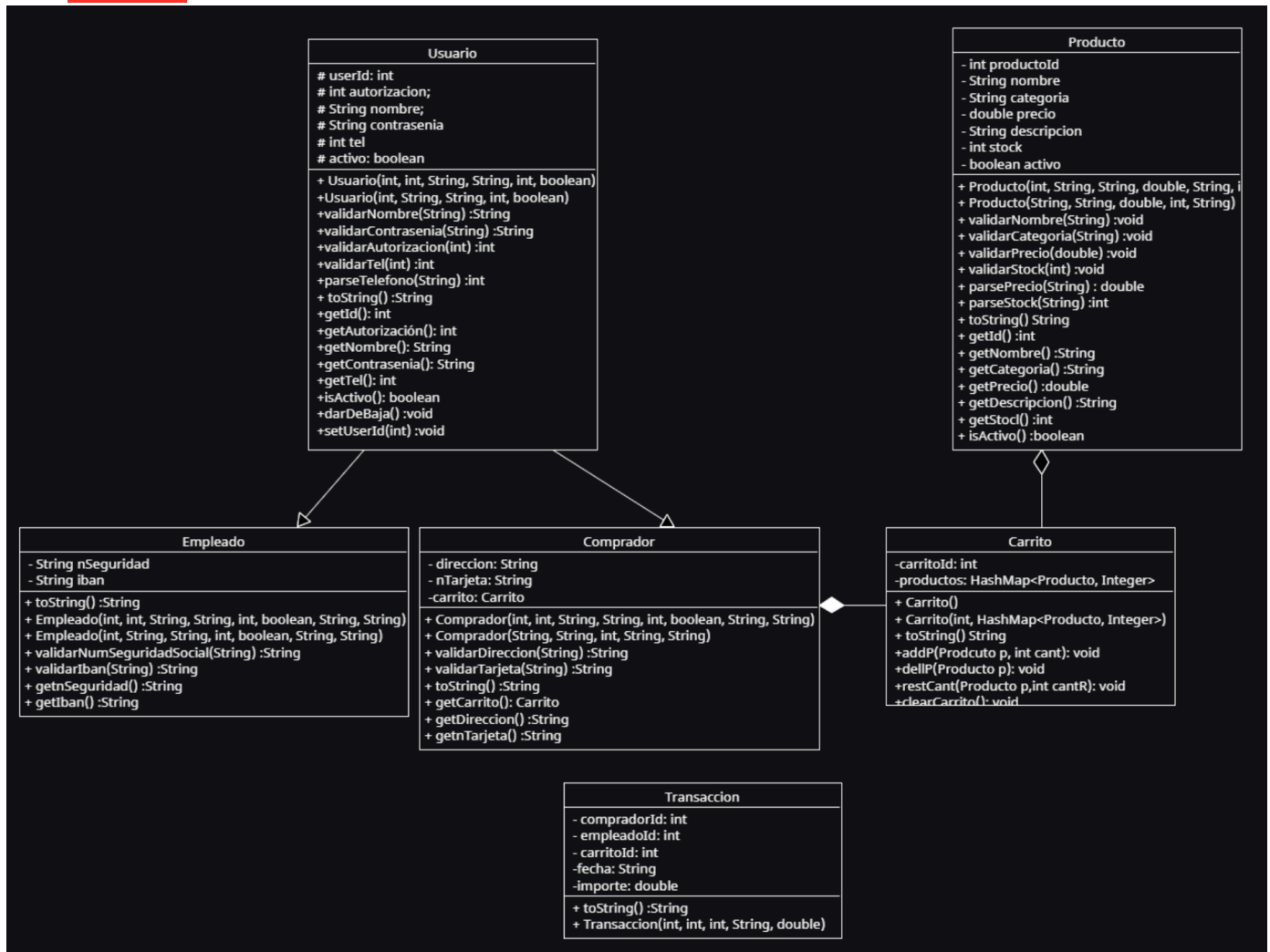
Caso de uso 5.1: "Pagar"

- Descripción: Registra la transacción en la base de datos
- Flujo principal:
 - Si fue ayudado por un empleado, seleccionarlo
 - Registrar transacción

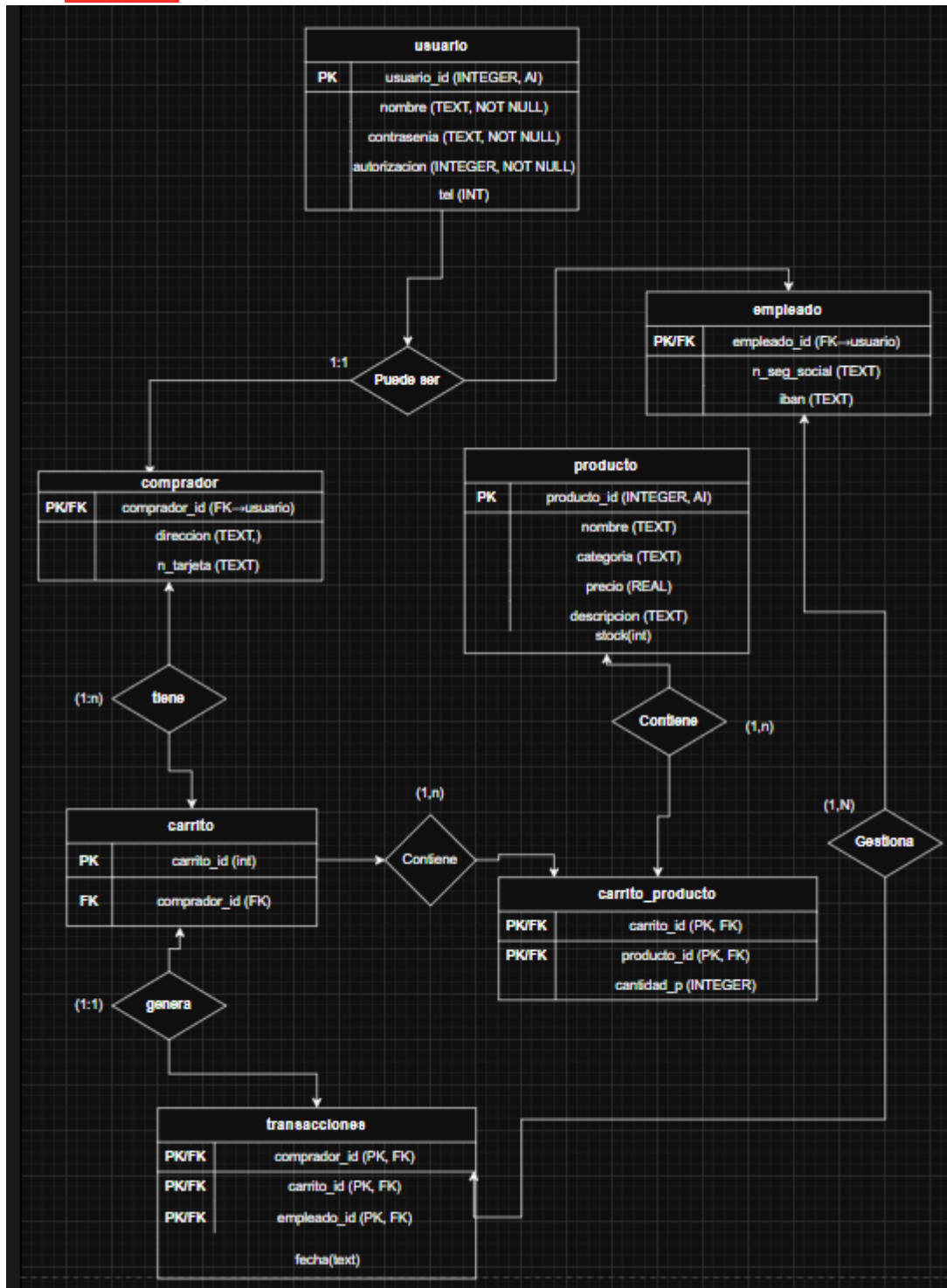
Caso de uso 5.2: Eliminar producto del carrito

- Descripción: Eliminar producto del carrito de compra
- Flujo principal:
 - Seleccionar producto a eliminar
 - Eliminar producto
- Flujo alternativo:
 - Producto no seleccionado → Mensaje de error





VI. Modelo de datos



Script de creación:

PRAGMA foreign_keys = ON;

DROP TABLE IF EXISTS transacciones;

DROP TABLE IF EXISTS carrito_producto;

DROP TABLE IF EXISTS carrito;

DROP TABLE IF EXISTS comprador;

DROP TABLE IF EXISTS empleado;



```
DROP TABLE IF EXISTS producto;  
DROP TABLE IF EXISTS usuario;
```

```
CREATE TABLE usuario(  
    usuario_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    autorizacion INTEGER NOT NULL CHECK (autorizacion IN (1, 2, 3)),  
    nombre TEXT NOT NULL,  
    contrasenia TEXT NOT NULL,  
    activo INTEGER NOT NULL DEFAULT 1 CHECK (activo IN (0, 1)),  
    tel INTEGER  
);
```

```
CREATE TABLE comprador(  
    comprador_id INTEGER,  
    direccion TEXT NOT NULL,  
    n_tarjeta TEXT NOT NULL,  
    CONSTRAINT fk_Co FOREIGN KEY (comprador_id) REFERENCES usuario(usuario_id),  
    CONSTRAINT pk_idComp PRIMARY KEY (comprador_id)  
);
```

```
CREATE TABLE carrito(  
    carrito_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    comprador_id INTEGER,  
    CONSTRAINT fk_idCar FOREIGN KEY(comprador_id) REFERENCES comprador(comprador_id)  
);
```

```
CREATE TABLE producto(  
    producto_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    nombre TEXT NOT NULL,  
    categoria TEXT NOT NULL,  
    precio REAL NOT NULL,  
    descripcion TEXT,  
    stock INTEGER NOT NULL DEFAULT 0,  
    activo INTEGER NOT NULL DEFAULT 1 CHECK (activo IN (0, 1))  
);
```

```
CREATE TABLE carrito_producto(  
    carrito_id INTEGER,  
    producto_id INTEGER,  
    cantidad_p INTEGER,  
    CONSTRAINT pk_idCaPr PRIMARY KEY (carrito_id, producto_id),  
    CONSTRAINT fk_idCa FOREIGN KEY (carrito_id) REFERENCES carrito(carrito_id),  
    CONSTRAINT fk_idPr FOREIGN KEY (producto_id) REFERENCES producto(producto_id)  
);
```

```
CREATE TABLE empleado(  
    empleado_id INTEGER,
```



```
n_seg_social TEXT,  
iban TEXT,  
CONSTRAINT fk_Em FOREIGN KEY (empleado_id) REFERENCES usuario(usuario_id),  
CONSTRAINT pk_Em PRIMARY KEY (empleado_id)  
);  
  
CREATE TABLE transacciones(  
    comprador_id INTEGER,  
    carrito_id INTEGER,  
    empleado_id INTEGER,  
    fecha TEXT NOT NULL DEFAULT (date('now','localtime')),  
    CONSTRAINT fk_Co FOREIGN KEY (comprador_id) REFERENCES comprador(comprador_id)  
        ON DELETE CASCADE ON UPDATE CASCADE,  
    CONSTRAINT fk_Ca FOREIGN KEY (carrito_id) REFERENCES carrito(carrito_id),  
    CONSTRAINT fk_Em FOREIGN KEY (empleado_id) REFERENCES empleado(empleado_id),  
    CONSTRAINT pk_CoCaEm PRIMARY KEY (comprador_id, carrito_id)  
);  
  
INSERT INTO usuario (autorizacion, nombre, contrasenia, activo, tel) VALUES  
    (1, 'admin', 'admin', 1, 600111111),  
    (1, 'Zoe', '1234', 1, 123456789),  
    (2, 'empleado', 'empleado', 1, 600222222),  
    (3, 'comprador', 'comprador', 1, 600333333);  
  
INSERT INTO empleado (empleado_id, n_seg_social, iban) VALUES  
    (1, '28/11111111/01', 'ES8023100001180000011111'),  
    (2, '123456789', 'ES1234567890123456789012'),  
    (3, '28/987654321/02', 'ES8023100001180000012345');  
  
INSERT INTO comprador (comprador_id, direccion, n_tarjeta) VALUES  
    (4, 'Calle Mayor 10, Madrid', '4111111111111111');  
  
INSERT INTO producto (nombre, categoria, precio, descripcion, stock, activo) VALUES  
    ('The Last of Us Part II', 'Videojuego', 49.99, 'PS4/PS5', 30, 1),  
    ('Sony WH-1000XM5', 'Audio', 349.99, 'Auriculares ANC', 12, 1),  
    ('Kindle Paperwhite', 'E-reader', 139.99, '32 GB, luz cálida', 25, 1),  
    ('Logitech MX Master 3S', 'Periférico', 99.99, 'Ratón inalámbrico', 20, 1),  
    ('Samsung 970 EVO 1TB', 'Almacenamiento', 89.99, 'NVMe M.2', 0, 1);  
  
INSERT INTO carrito (comprador_id) VALUES (4);
```



```
INSERT INTO carrito_producto (carrito_id, producto_id, cantidad_p) VALUES  
(1, 1, 1),  
(1, 4, 1);
```

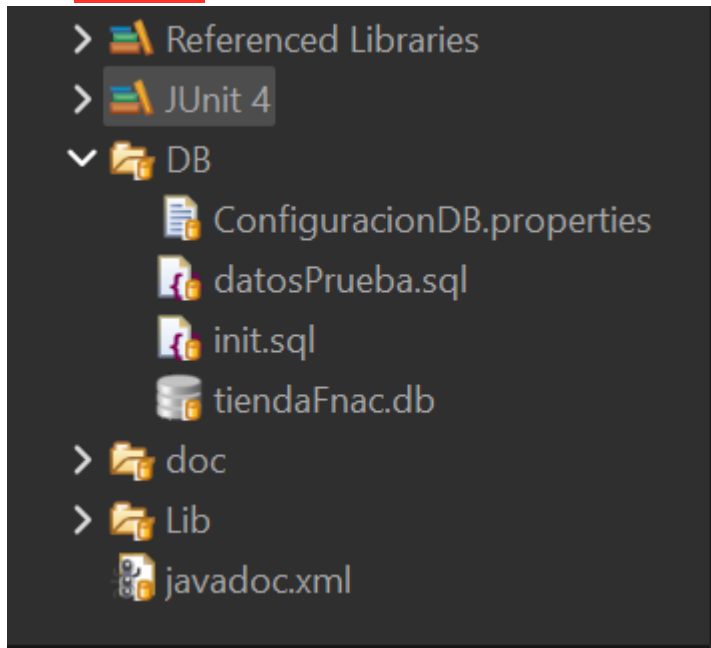
```
INSERT INTO transacciones (comprador_id, carrito_id, empleado_id) VALUES  
(4, 1, 3);
```

VII. Arquitectura de la aplicación

- Explicación del patrón MVC:
 - Modelo → Clases representantes de los datos y acceso a la base de datos
 - Pojos: Clases representantes de las entidades.
 - Accessbd: Clases encargadas de gestionar la conexión con la base de datos
 - Vista → Clases representantes de la interfaz gráfica
 - Controlador → Intermediario entre el modelo y la vista. Fija el comportamiento de la aplicación
- Tecnologías usadas:
 - Lenguaje de programación: Java
 - Sistema gestor de base de datos: Sqlite
 - IDE utilizado: Eclipse



- ▼ > GestionTienda [TrabajoIntrigrador zc
- > JRE System Library [JavaSE-21]
- ▼ > src
 - ▼ com.dam.ctrl
 - > Ctrl.java
 - ▼ com.dam.main
 - > App.java
 - ▼ com.dam.model.acesbd
 - > AccessDBProp.java
 - > TableCarritoDAO.java
 - > TableCarritoProductoDAO.java
 - > TableProductoDAO.java
 - > TableTransaccionesDAO.java
 - > TableUsuarioDAO.java
 - ▼ > com.dam.model.pojos
 - > > Carrito.java
 - > Comprador.java
 - > Empleado.java
 - > Producto.java
 - > Transacciones.java
 - > Usuario.java
 - ▼ com.dam.pruebas
 - > CarritoAgregarProductoTest.java
 - > CarritoEliminarProductoTest.java
 - > EmpleadoAltaTest.java
 - > EmpleadoEliminacionTest.java
 - > ProductoAltaTest.java
 - > ProductoConsultaTest.java
 - > ProductoEliminacionTest.java
 - > ProductoModificacionTest.java
 - > SimulacroPagoTest.java
 - > StockAgregarTest.java
 - > StockEliminarTest.java
 - > TableCarritoDAOTest.java



VIII. Equipo

David Martinez Gallego→Scrum master, desarrollador

- Crea las tareas a realizar en jira
- Participa en el desarrollo del código de la aplicación

Miguel Gordo→Documentación,QA

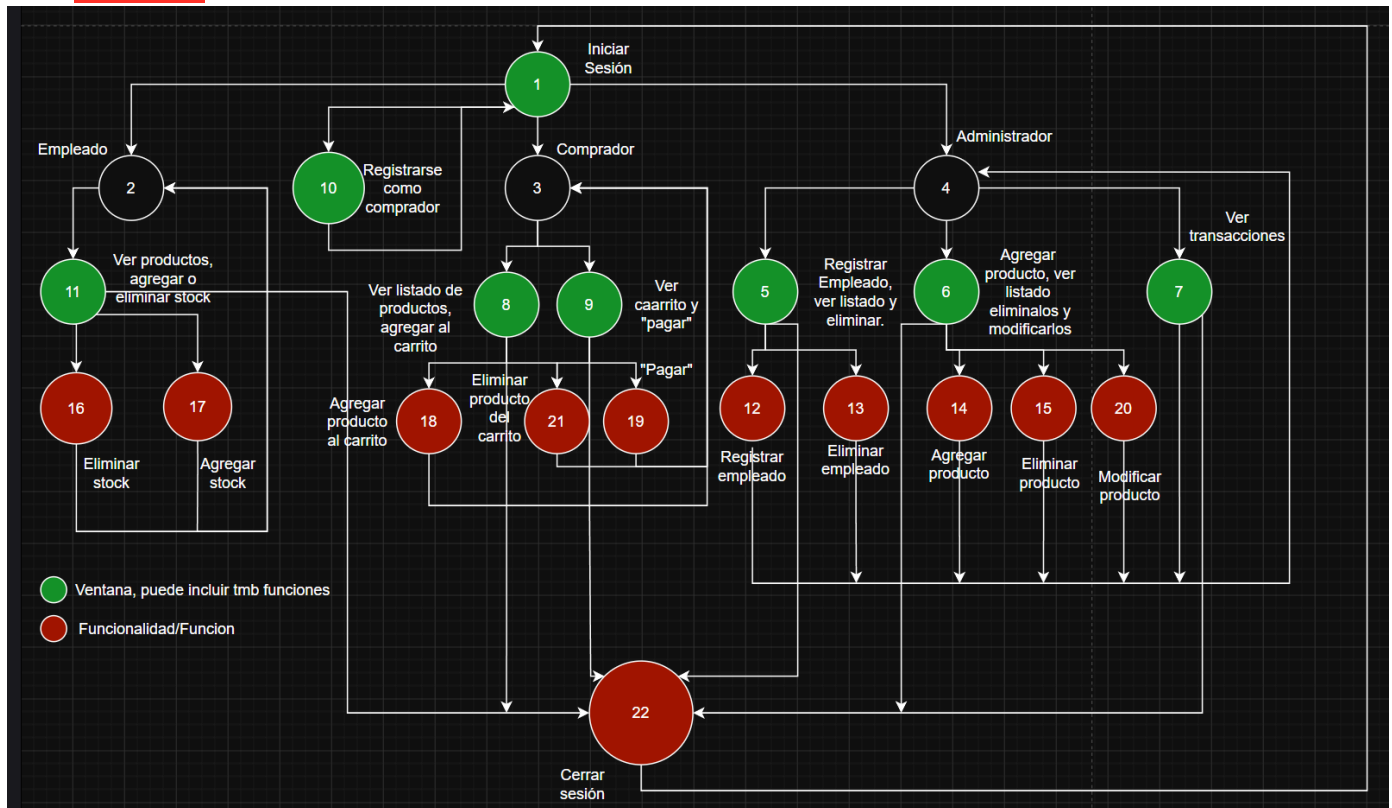
- Crea la documentación necesaria para presentar la aplicación
- Genera las pruebas y test pertinentes para probar el correcto funcionamiento de la aplicación

Zoe Jastreby→Cloud master, desarrollador

- Manejo del repositorio de git.
- Participa en el desarrollo del código de la aplicación

IX. Observaciones

- Tanto al eliminar stock como al agregar productos al carrito, la aplicación limitará la cantidad introducida por el usuario al stock disponible de cada producto, evitando así valores fuera de rango.
- A continuación paso a dejar nuestro grafo de flujo que es una guía para todos los diagramas.



- Ya que el proyecto sigue en desarrollo esta documentación irá fluctuando con él a medida que avance hasta su finalización.
- Nuestro jira: <https://zoejastre.atlassian.net/jira/software/projects/TPI/boards/1/backlog?atlOrigin=eyJpIjoiOTZINTNjMGU2OWU2NDI2NDIIZTU2MDcwYTEwYjY1MWMiLCJwIjoiajI9>
- Nuestro repositorio de gitHub: <https://github.com/Pandakira25/TrabajoIntrador>